

EazyEats: A Canteen at your Fingertips



A

Project Report

Submitted in partial fulfillment of the requirement for the award of degree of

Bachelor of Technology

In

Information Technology

Submitted to

**RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA,
BHOPAL (M.P.)**

Guided By

Prof. Pawan K. Gupta

Submitted By

Mauli Patel(0827IT221091)

Palak Sanjay Gahile(0827IT221103)

Shruti Sharma(0827IT221136)

Sneha Shrivastava (0827IT221137)

**DEPARTMENT OF INFORMATION TECHNOLOGY
ACROPOLIS INSTITUTE OF TECHNOLOGY & RESEARCH,
INDORE (M.P.) 453771
2025-2026**

Declaration

I hereby declared that the work, which is being presented in the project entitled **EazyEats: A Canteen at your Fingertips** partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology**, submitted in the department of Information Technology at **Acropolis Institute of Technology & Research, Indore** is an authentic record of my own work carried under the supervision of “**Prof. Pawan K. Gupta**”. I have not submitted the matter embodied in this report for the award of any other degree.

Mauli Patel(0827IT221091)

Palak Sanjay Gahile(0827IT221103)

Shruti Sharma(0827IT221136)

Sneha Shrivastava (0827IT221137)

Prof. Pawan K. Gupta

Supervisor

Project Approval Form

I hereby recommend that the project **EazyEats: A Canteen at your Fingertips** prepared under my supervision by **Mauli Patel (0827IT221091), Palak Sanjay Gahile(0827IT22103), Shruti Sharma(0827IT221136), Sneha Shrivastava (0827IT221137)** be accepted in partial fulfillment of the requirement for the degree of Bachelor of Technology in Information Technology.

Prof. Pawan K. Gupta

Supervisor

Recommendation concurred in 2025-2026

Prof. Shahida Khan

Project Incharge

Prof. Pawan K. Gupta

Project Coordinator

Acropolis Institute of Technology & Research

Department of Information Technology



Certificate

The project work entitled **EazyEats: A Canteen at your Fingertips** submitted by **Mauli Patel(0827IT221091),Palak Sanjay Gahile (0827IT221103),Shruti Sharma(0827IT221136), Sneha Shrivastava (0827IT221137)** is approved as partial fulfillment for the award of the degree of Bachelor of Technology in Information Technology by Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.).

Internal Examiner

Name:.....

Date:/.../.....

External Examiner

Name:

Date:/.../.....

Acknowledgement

With boundless love and appreciation, we would like to extend our/my heartfelt gratitude and appreciation to the people who helped us to bring this work to reality. We would like to have some space of acknowledgement for them.

To the project in-charge **Prof. Shahida Khan** and project coordinator **Prof. Pawan K. Gupta** for their constructive comments, suggestions, and critiquing even in hardship.

To the honorable **Prof. (Dr.) Prashant Lakkadwala**, Head, Department of Information Technology for his favorable responses regarding the study and providing necessary facilities.

To the honorable **Dr. S.C. Sharma**, Director, AITR, Indore for his unending support, advice and effort to make it possible.

Finally, we would like to pay my/our thanks to faculty members and staff of the Department of Computer Science & Engineering for their timely help and support.

We also like to pay thanks to our **parents** for their eternal love, support and prayers without them it is not possible.

Mauli Patel(0827IT221091)

Palak Sanjay Gahile(0827IT221103)

Shruti Sharma(0827IT221136)

Sneha Shrivastava (0827IT221137)

Abstract

EazyEats: A Canteen at Your Fingertips is a cloud-based food ordering system created to solve common problems in cafeterias such as long queues, delayed service, and mismanagement of orders. Many schools, colleges, offices, hospitals, and event venues face these issues daily, which lead to customer dissatisfaction, food wastage, and loss in sales. EazyEats was designed to make the food ordering process faster, smoother, and more efficient for both customers and cafeteria staff. This system allows users to pre-order their meals through a web or mobile application, choose a convenient pick-up time, and pay online using UPI, debit/credit cards, or wallets. On the other side, cafeteria staff can use an admin dashboard to manage orders, update the menu, track inventory, and receive instant notifications about new orders. The system uses cloud technology, built with tools like React.js for the front end and Node.js for the back end. It also includes payment integration using Razorpay and a real-time database like Firebase or MongoDB. The project found that by using this system, food services can become much more organized. It reduces waiting time for customers, cuts down food waste, and helps the staff work more efficiently. Customers also benefit from a smoother and more convenient ordering experience. Overall, EazyEats brings modern digital solutions to everyday cafeteria problems. It improves service quality, increases customer satisfaction, and helps manage operations better. This system can be a useful and scalable solution for many different places where food is served to a large number of people.

Table of Content

Declaration	II
Project Approval Form.....	III
Acknowledgement	IV
Abstract	V
List of Figures... ..	X
List of Tables... ..	XI
Abbreviations.....	XII
Chapter 1: Introduction	
1.1 Rationale	1
1.2 Existing System.....	1
1.3 Problem Formulation.....	2
1.4 Proposed System	3
1.5 Objectives.....	3
1.6 Contribution of the Project.....	4
1.6.1 Market Potential	4
1.6.2 Innovativeness	4
1.6.3 Usefulness	4
1.7 Report Organization	5
Chapter 2: Requirement Engineering	
2.1 Feasibility Study (Technical, Economical, Operational)	7
2.2 Requirement Collection.....	8
2.2.1 Discussion	8

2.2.2 Requirement Analysis	8
2.3 Requirements.....	8
2.3.1 Functional Requirements.....	8
2.3.1.1 Statement of Functionality	9
2.3.2 Nonfunctional Requirements.....	9
2.3.2.1 Statement of Functionality	9
2.4 Hardware & Software Requirements	9
2.4.1 Hardware Requirement (Developer & End User)	9
2.4.2 Software Requirement (Developer & End User)	10
2.5 Use-case Diagrams.....	11
2.5.1 Use-case Descriptions	11
Chapter 3: Analysis & Conceptual Design & Technical Architecture	
3.1 Technical Architecture	14
3.2 Sequence Diagrams.....	15
3.3 Class Diagrams.....	16
3.4 DFD.....	17
3.5 User Interface Design.....	18
3.6 Data Design.....	19
3.6.1 Schema Definitions	19
3.6.2 E-R Diagram	23
Chapter 4: Implementation & Testing	
4.1 Methodology	25
4.1.1 Proposed Algorithm	25
4.2 Implementation Approach.....	26

4.2.1 Introduction to Languages, IDEs Tools and Technologies	26
4.3 Testing Approaches	27
4.3.1 Unit Testing.....	27
a. Test Cases.....	27
4.3.2 Integration Testing	27
b. Test Cases.....	27

Chapter 5: Results & Discussion

5.1 User Interface Representation	29
5.1.1 Brief Description of Various Modules	29
5.2 Snapshot of System with Brief Description	30
5.3 Database Description.....	34
5.3.1 Snapshot of Database Tables with Brief Description.....	35
5.4 Final Findings.....	36

6. Conclusion & Future Scope

6.1 Conclusion.....	38
6.2 Future Scope.....	38

REFERENCES

Appendix A: Project Synopsis

Appendix B: Guide Interaction Report

Appendix C: User Manual

Appendix D: Git/GitHub Commits/Version History

List of Figures

1.	Fig.1.Use-Case.....	11
2.	Fig.2. Technical Architecture	14
3.	Fig.3.Sequence Diagrams	15
4.	Fig.4. Class Diagrams.....	16
5.	Fig.5.Data Flow Diagram	17
6.	Fig.6.Login Page.....	18
7.	Fig.6.1. User Interface Design.....	18
8.	Fig.6.2 Menu Page.....	19
9.	Fig.7. Schema Definations.....	19
10.	Fig.8. ER Diagram.....	23
11.	Fig.9.Login Page.....	31
12.	Fig.10.User Dashboard	31
13.	Fig.11.Menu Page.....	32
14.	Fig.12. Add to Cart	32
15.	Fig.13.Order Summary	33
16.	Fig.14.Order Tracking	33

List of Tables

1	TABLE 1 PROBLEM FORMULATION	2
2	TABLE 2 TOOLS & TECHNOLOGIES	26
3	TABLE 3 TEST CASES	27
4	TABLE 4 TEST CASES	27
5	TABLE 5 MODULES	29
6	TABLE 6 DATABASE	35

Abbreviations

1. RDBMS - Relational Database Management System
2. AWS SES - Amazon Web Services
3. API - Application Programming Interface
4. UI - User Interface
5. IDE - Integrated Development Environment
6. HTML - Hypertext Markup Language
7. CSS - Cascading Style Sheets
8. OS-Operating System
9. DBMS-Database Management System
10. PWA-Progressive Web App
11. DFD-Data Flow Diagram
12. ER-Entity-Relationship
13. JS-JavaScript

Chapter 1:Introduction

Introduction

1.1 Rationale

In today's fast-paced environments like educational institutions, corporate offices, hospitals, and the event venues, cafeterias face numerous challenges such as long queues, order mismanagement, and delays in service. With increasing demand for convenience and efficiency, traditional food ordering systems no longer meet user expectations. Hence, there is a pressing need for a digital, cloud-based solution that simplifies and automates the cafeteria food ordering process.

EazyEats is designed to offer a seamless, efficient, and user-friendly experience by enabling pre-orders, digital payments, real-time tracking, and efficient inventory and order management.

1.2 Existing System

Existing food ordering platforms such as FoodPanda, Grubhub, and MyCafe provide general food delivery services but lack specific customization and integration for institutional cafeterias. They face issues like:

- 1.2.1 Delivery delays
- 1.2.2 Hidden or high charges
- 1.2.3 Limited availability and customization options
- 1.2.4 Poor integration with real-time inventory or staff workflows

Additionally, current systems used in canteens often involve manual order taking and billing, leading to human errors, slow service, and food wastage.

1.3 Problem Formulation

The major challenges faced in traditional cafeteria systems include:

- 1.3.1 Long queues and wait times during peak hours
- 1.3.2 Miscommunication and errors in order preparation
- 1.3.3 Inefficient inventory and menu management
- 1.3.4 Lack of real-time data on customer preferences and sales
- 1.3.5 No facility for advance scheduling or digital payment

There are many existing systems for this topic but have certain limitations and problems related to it. Such portals and problems related to it are mentioned below.

TABLE 1 PROBLEM FORMULATION

Sr. No.	Name of Existing	Features	Limitations
1.	FoodPanda	1. Real-time order ng. 2. Wide restaurant ion. 3. User-friendly interface.	1. Delivery delays 2. Delivery charges 3. Limited bility
2.	Grubhub	1. Rewards program 2. Promotions ,loyalty 3. Technology ations	1. Delivery delays 2. Hidden fees 3. Limited mization

3.	MyCafe	1. Responsive design 2. Simple visuals and UI 3. Static web pages	1. Manual order ment 2. No login/signup 3. No feedback or system
----	--------	---	--

1.4 Proposed System

To address cafeteria congestion and inefficiencies, EazyEats proposes a cloud-based food ordering system with the following solutions:

1.4.1 Pre-Order & Scheduling – Users can place meal orders in advance and select a preferred pick-up time.

1.4.2 Digital Payments – Integrates Razorpay for seamless cashless transactions.

1.4.3. Real-Time Order Tracking – Users can monitor order status, reducing uncertainty.

1.4.4. Admin Dashboard – Cafeteria staff can manage menus, track inventory, and receive order notifications.

1.4.5. Mobile & Web Access – Ensures convenient food ordering from any device.

1.5 Objectives

1.5.1 Develop a cloud-based platform for users to pre-order meals, reducing wait times and improving efficiency.

1.5.2 Integrate real-time order tracking and digital payment options for a seamless ordering experience.

1.5.3 Provide an admin dashboard for cafeteria staff to manage menus, inventory, and order fulfillment efficiently.

1.6 Contribution of the Project

1.6.1 Market Potential

1.6.1.1 Universities and colleges with thousands of students.

1.6.1.2 Corporate offices aiming for efficient lunch management.

1.6.1.3 Hospitals where contactless food service is essential.

1.6.1.4 Event venues and food courts handling large crowds.

1.6.1.5 This platform can be scaled to multiple cafeterias

1.6.2 Innovativeness

EazyEats stands out because of:

1.6.2.1 Pre-order and scheduling functionality to avoid crowding.

1.6.2.2 Real-time inventory and order tracking

1.6.2.3 A dedicated admin dashboard for cafeteria staff

1.6.2.4 Seamless integration with digital payment gateways

1.6.2.5 A cloud-hosted infrastructure for better scalability and access

1.6.3 Usefulness

The platform benefits multiple stakeholders:

1.6.3.1 Students/Employees: Quick and convenient food ordering

1.6.3.2 Cafeteria Staff: Easier order management and reduced chaos

1.6.3.3 Management: Real-time reports for better decision-making

1.6.3.4 Environment: Reduction in food waste through accurate demand prediction

1.7 Report Organization

- **Chapter 2: Literature Survey** – Reviews existing systems, their features, and limitations, and compares them with EazyEats.
- **Chapter 3: Theoretical Analysis** – Discusses the architecture, system requirements, and block diagram.
- **Chapter 4: Application** – Describes various sectors where EazyEats can be implemented.

- **Chapter 5: Methodology** – Details the development process, including technology stack and system workflow.
- **Chapter 6: Testing and Evaluation** – Explains testing methods and system validation.
- **Chapter 7: Results and Discussion** – Presents the outcome, user feedback, and performance.
- **Chapter 8: Conclusion and Future Work** – Summarizes findings and suggests future enhancements

Chapter2:Requirement Engineering

Requirement Engineering

2.1 Feasibility Study (Technical, Economical, Operational)

2.1.1 Technical Feasibility

2.1.1.1 The system uses modern web technologies such as React.js, Node.js, Firebase, and Razorpay for payment integration.

2.1.1.2 Cloud-hosted on AWS for high availability and real-time accessibility.

2.1.1.3 Technologies used are reliable, scalable, and widely adopted.

2.1.1.4 Availability of skilled developers and open-source resources ensures ease of implementation.

2.1.1.5 Technical setup supports both web and mobile platforms.

2.1.2 Economic Feasibility

2.1.2.1 Uses open-source tools and cloud platforms with free tiers (e.g., Firebase, MongoDB), minimizing infrastructure costs.

2.1.2.2 AWS credits for students help reduce initial deployment expenses.

2.1.2.3 Long-term cost savings expected due to:

- Reduced manual labor.
- Minimized food wastage.
- Improved efficiency in order processing.

2.1.3 Operational Feasibility

2.1.3.1 User-friendly interfaces designed for both end users and administrators.

2.1.3.2 Minimal training is required for staff to operate the system effectively.

2.1.3.3 Features such as real-time notifications and admin dashboards simplify cafeteria operations.

2.1.3.4 Solves key issues like:

- Long queues.
- Delayed service.

- Inefficient menu updates.

2.2 Requirement Collection

2.2.1 Discussion

Stakeholders such as students, cafeteria staff, and administrative personnel were consulted. Discussions revealed frustration with long queues, payment issues, and poor menu visibility. Staff highlighted challenges in inventory management and manual order handling.

2.2.2 Requirement Analysis

Based on stakeholder feedback, the following user needs were identified:

2.2.2.1 Fast and convenient food ordering.

2.2.2.2 Real-time tracking of order status.

2.2.2.3 Seamless and secure digital payments.

2.2.2.4 Efficient backend for inventory and menu management.

2.2.2.5 These needs were categorized into functional and nonfunctional requirements

2.3 Requirements

2.3.1 Functional Requirements

2.3.1.1 Statement of Functionality

- **User Registration/Login:** Users can register and securely log in.
- **Menu Browsing:** Users can view and select meals from the digital menu.
- **Pre-ordering:** Users can schedule orders for later pickup.
- **Digital Payments:** Users can pay via Razorpay using UPI, credit/debit cards, or wallets.
- **Order Tracking:** Users receive real-time updates on order status.
- **Admin Dashboard:** Staff can manage menus, update inventory, view sales, and receive order notifications.

- **Inventory Management:** Real-time updates help reduce food wastage.

2.3.2 Nonfunctional Requirements

2.3.2.1 Statement of Functionality

- **Performance:** The system must process orders in real-time with minimal latency.
- **Scalability:** The application should handle a growing number of users and transactions.
- **Reliability:** Ensure system availability during peak hours with cloud redundancy.
- **Security:** Secure user data and transaction details through HTTPS and secure APIs.
- **Usability:** The UI must be intuitive and responsive on mobile and desktop devices.
- **Maintainability:** Code and architecture should support future upgrades.

2.4 Hardware & Software Requirements

2.4.1 Hardware Requirement

2.4.1.1 Developer Side

- **Processor:** 2.5 GHz Quad-Core or higher
- **RAM:** 16 GB or higher
- **Storage:** 1 TB SSD
- **OS:** Windows
- **Network:** High-speed internet

2.4.1.2 End User Side

- **Smartphone or PC**
- **Processor:** 1.5 GHz Dual-Core
- **RAM:** 4 GB
- **Storage:** 256 GB SSD

- Browser: Google Chrome, Firefox, or Edge
- Network: Internet connection (Wi-Fi or 4G)

2.4.2 Software Requirement

2.4.2.1 Developer Side

- OS: Windows 10
- Frontend: React.js, HTML, CSS, JavaScript
- Backend: Node.js with Express.js
- Database: Firebase Firestore / MongoDB / MySQL
- Cloud: AWS (hosting & deployment)
- Payment Gateway: Razorpay
- IDE: VS Code
- Version Control: Git & GitHub

2.4.2.2 End User Side

- Browser: Chrome/Firefox/Safari
- OS: Android/iOS (for mobile access) or any modern OS (for web)
- App Access: Web browser (or PWA/mobile app)

2.5 Use-case Diagrams

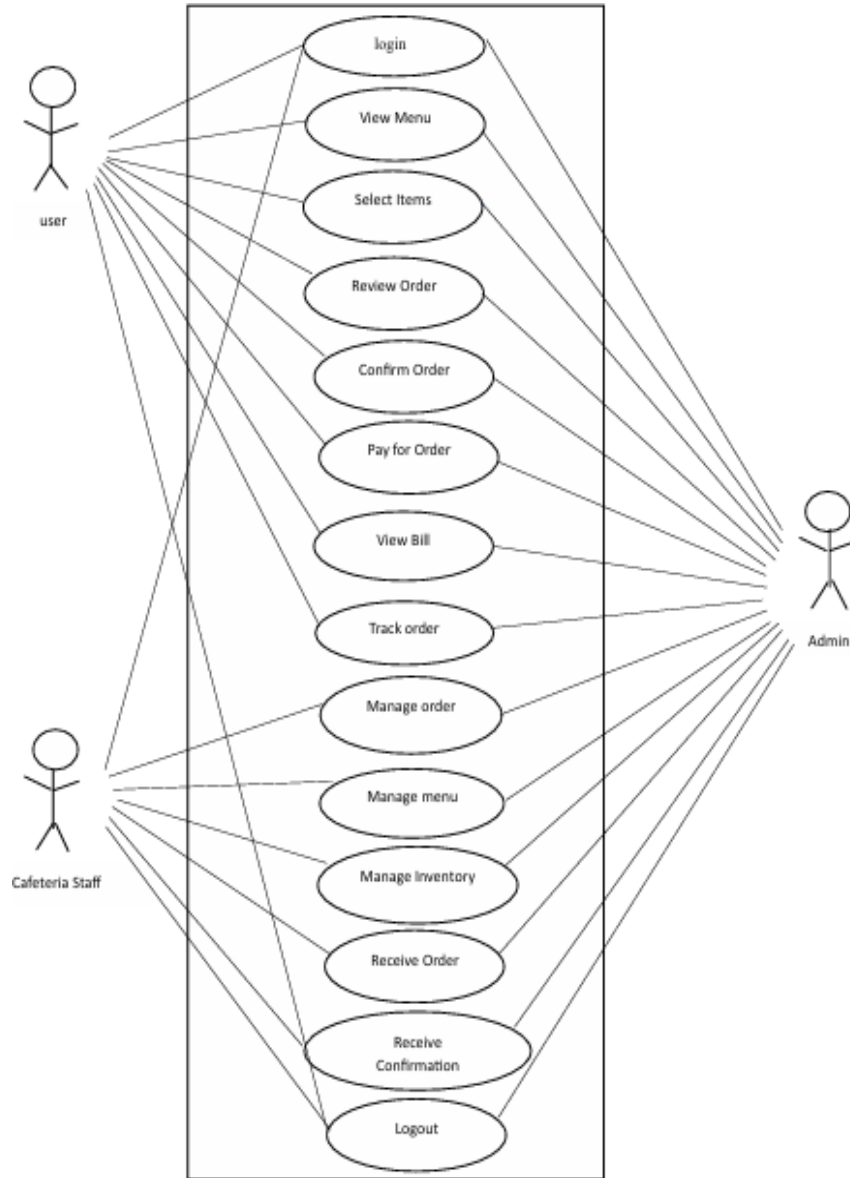


Fig 1. Use-Case

2.5.1 Use-case Descriptions

- **User Interaction:** Users can log in, view the menu, select items, review and confirm their orders, pay for the order, view bills, and track their orders. These are typical customer-facing functionalities to facilitate a seamless ordering experience.

- **Cafeteria Staff Role:** Cafeteria staff can log in, manage orders, receive orders, receive confirmations, manage the menu, and handle inventory. Their role supports the back-end operations of processing and fulfilling user orders.
- **Admin Capabilities:** Admins have the most comprehensive access. They can perform all user and cafeteria staff operations, in addition to managing users, overseeing inventory, and configuring system-level functionalities.
- **Order Management Process:** The system follows a structured order process: Users place an order → Cafeteria staff receive and confirm it → Staff update the system on completion → Users can track their orders and view confirmations.
- **System Boundaries:** The system boundary box indicates that all listed functionalities (use cases) are part of a centralized food ordering and management system, accessible by three actors: User, Cafeteria Staff, and Admin.

Chapter3: Analysis & Conceptual Design & Technical Architecture

Analysis & Conceptual Design & Technical Architecture

3.1 Technical Architecture

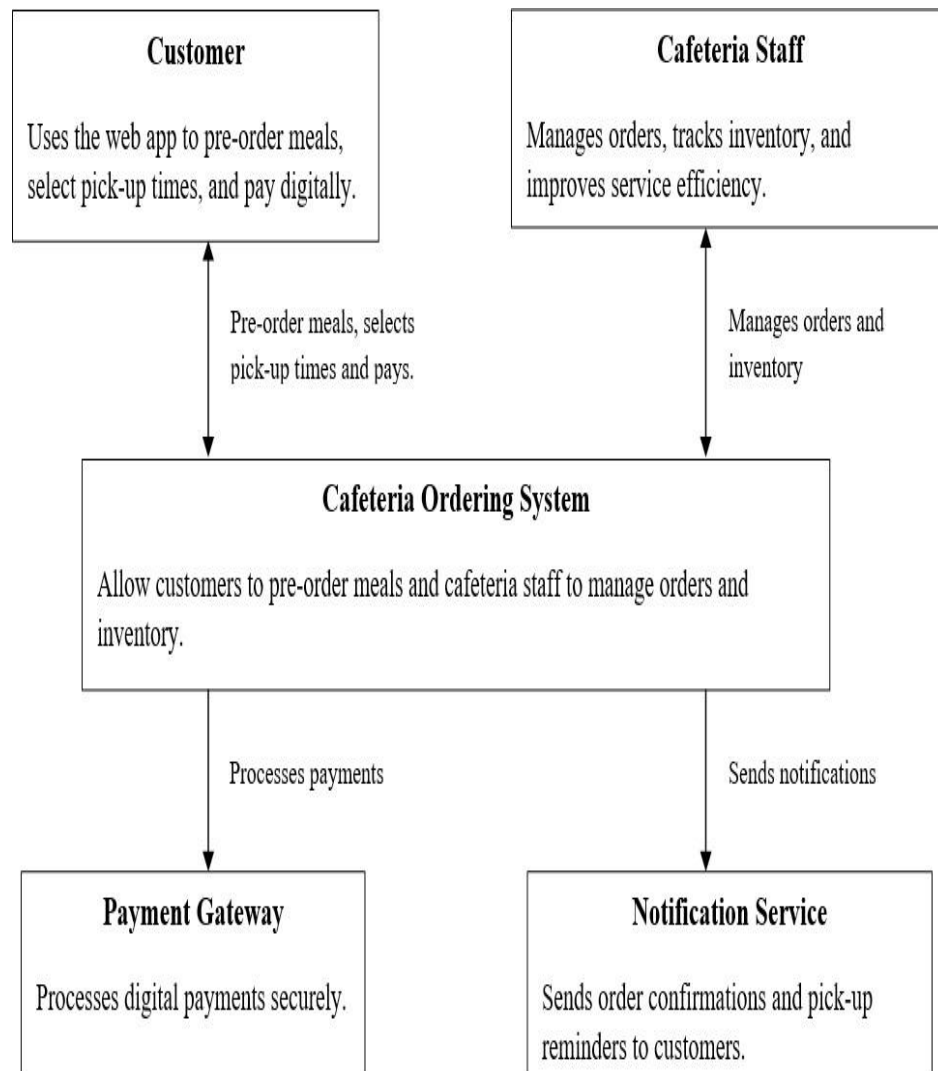


Fig.2. Technical Architecture

3.2 Sequence Diagrams

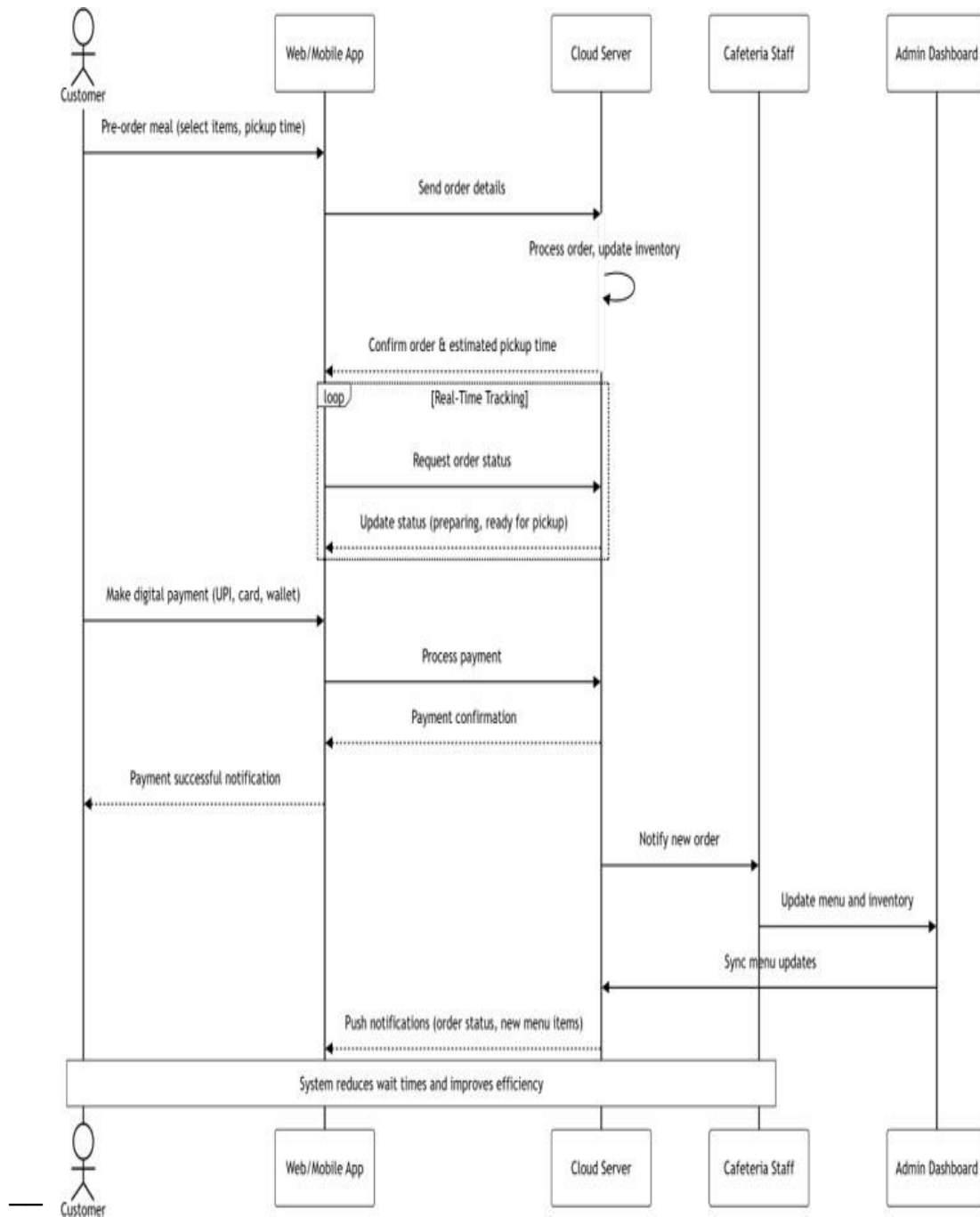


Fig.3. Sequence Diagrams

3.3 Class Diagrams

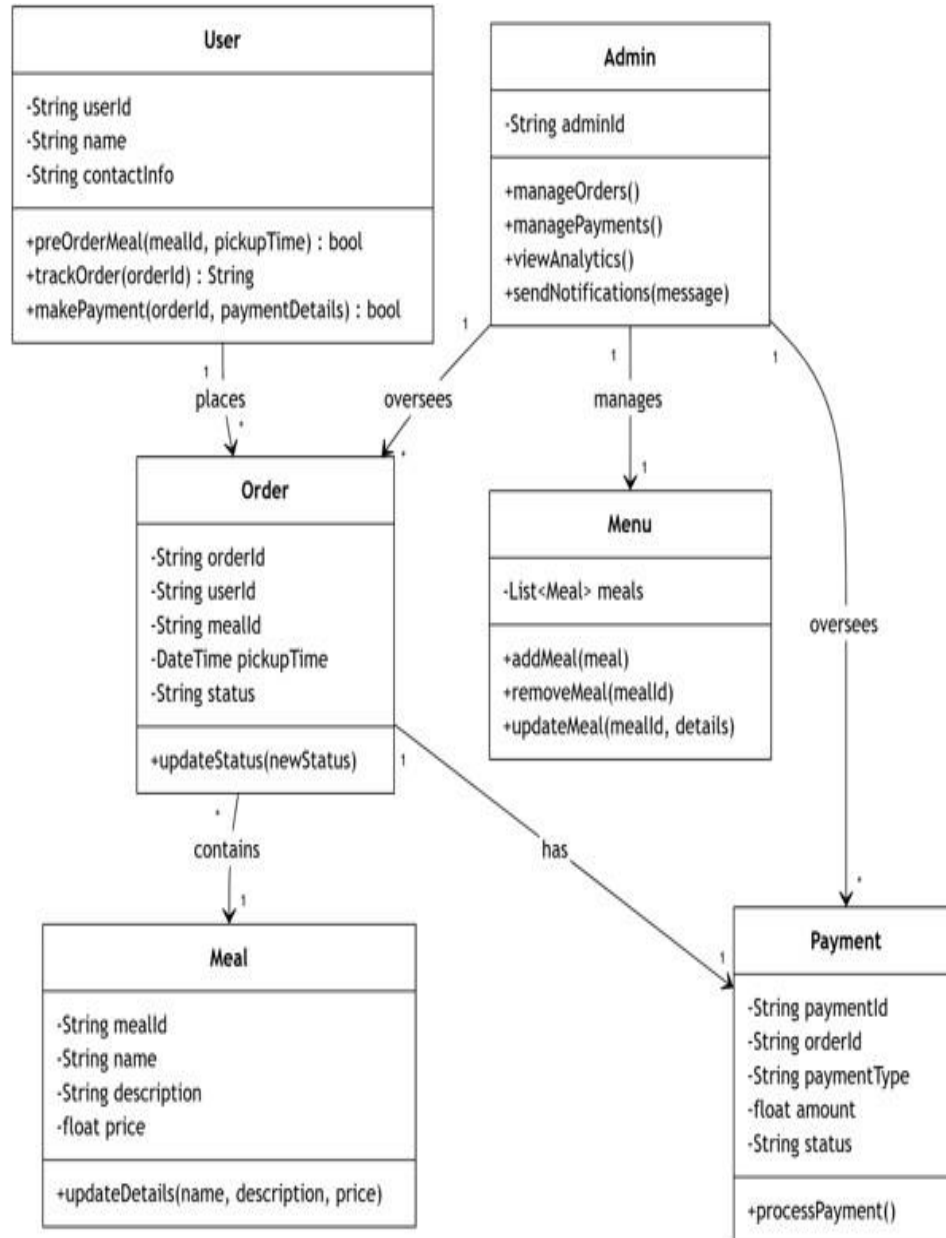


Fig.4. Class Diagrams

3.4 DFD

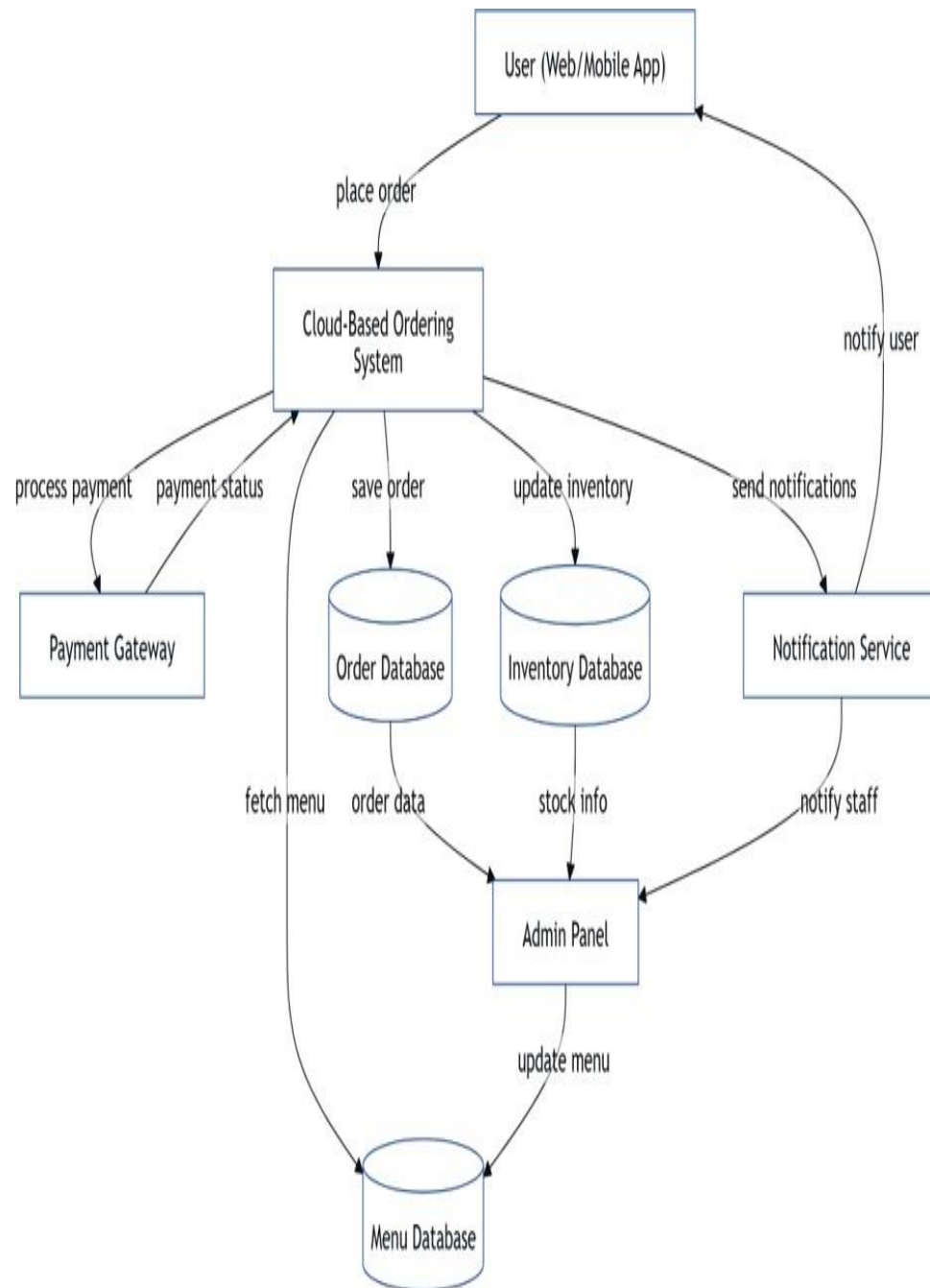


Fig.5. Data Flow Diagram

3.5 User Interface Design

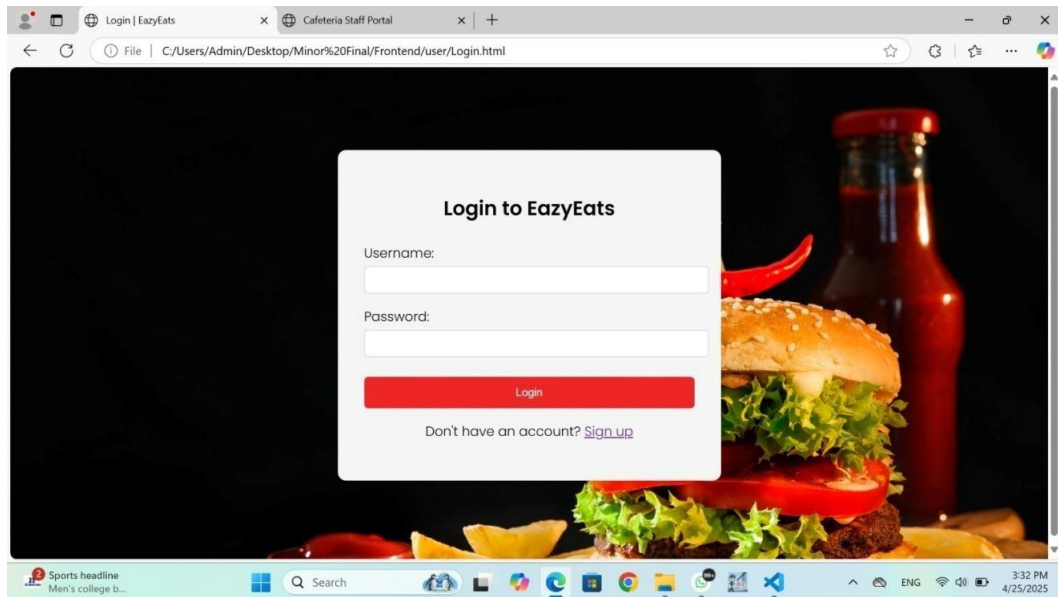


Fig.6. Login Page

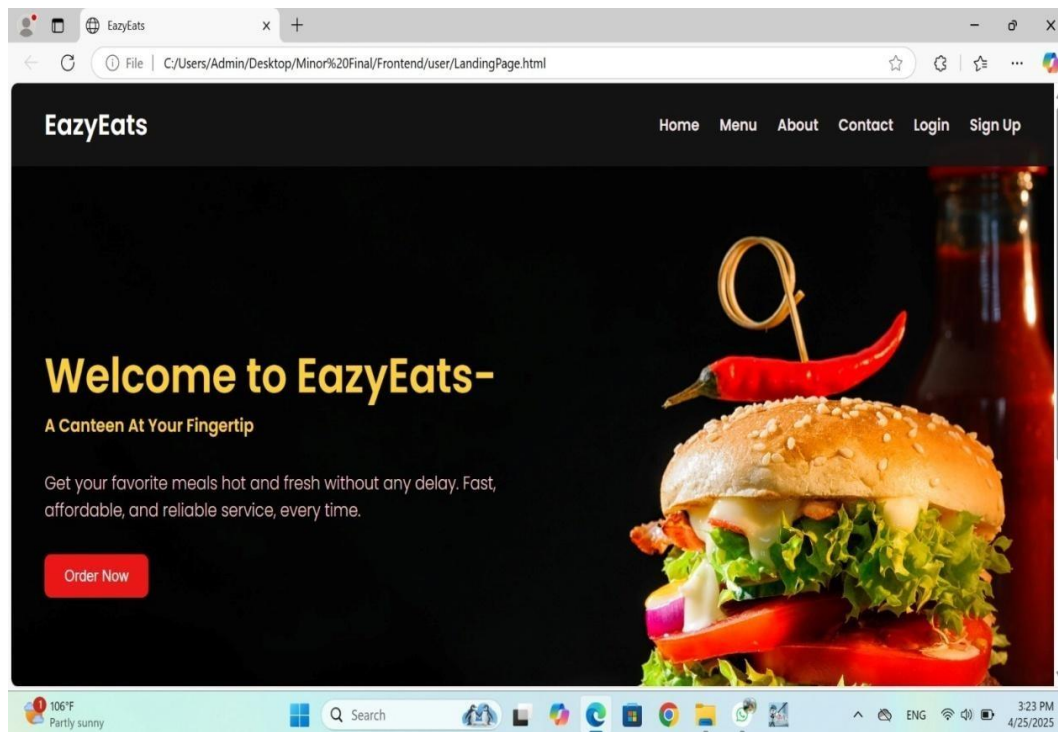


Fig.6.1 User Interface Design

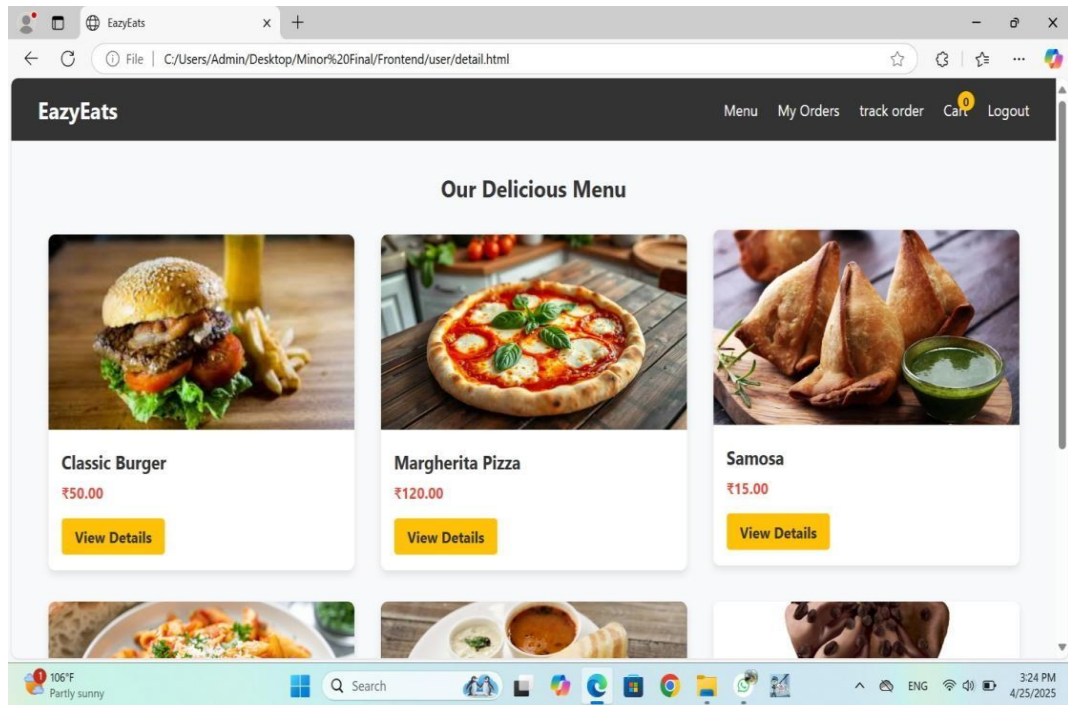


Fig.6.2.Menu Page

3.6 Data Design

3.6.1 Schema Definitions

```
{
  _id: ObjectId,
  name: String,
  email: String,
  phone: String,
  passwordHash: String,
  role: { type: String, enum: ["customer", "admin"], default: "customer" },
  createdAt: Date,
  updatedAt: Date
}
```



```
{
  _id: ObjectId,
  userId: ObjectId,
  message: String,
  type: { type: String, enum: ["order", "menu", "system"] },
  isRead: { type: Boolean, default: false },
  createdAt: Date
}
```

```
{
  _id: ObjectId,
  name: String,
  description: String,
  price: Number,
  category: String,
  imageUrl: String,
  isAvailable: Boolean,
  inventoryQuantity: Number,
  createdAt: Date,
  updatedAt: Date
}
```

```
{
  _id: ObjectId,
  userId: ObjectId,
  orderId: ObjectId,
  amount: Number,
  method: String,
  status: { type: String, enum: ["success", "failed", "pending"], default: "pending" },
  transactionId: String,
  timestamp: Date
}
```

```
{
  _id: ObjectId,
  userId: ObjectId,
  items: [
    {
      menuItemId: ObjectId,
      name: String,
      quantity: Number,
      price: Number
    }
  ],
  totalAmount: Number,
  status: { type: String, enum: ["preparing", "ready", "picked-up", "cancelled"], default: "pending" },
  pickupTime: Date,
  paymentStatus: { type: String, enum: ["pending", "paid", "failed"], default: "pending" },
  paymentMethod: String,
  createdAt: Date,
  updatedAt: Date
}
```

Fig.7. Schema Definitions

3.6.2 E-R Diagram

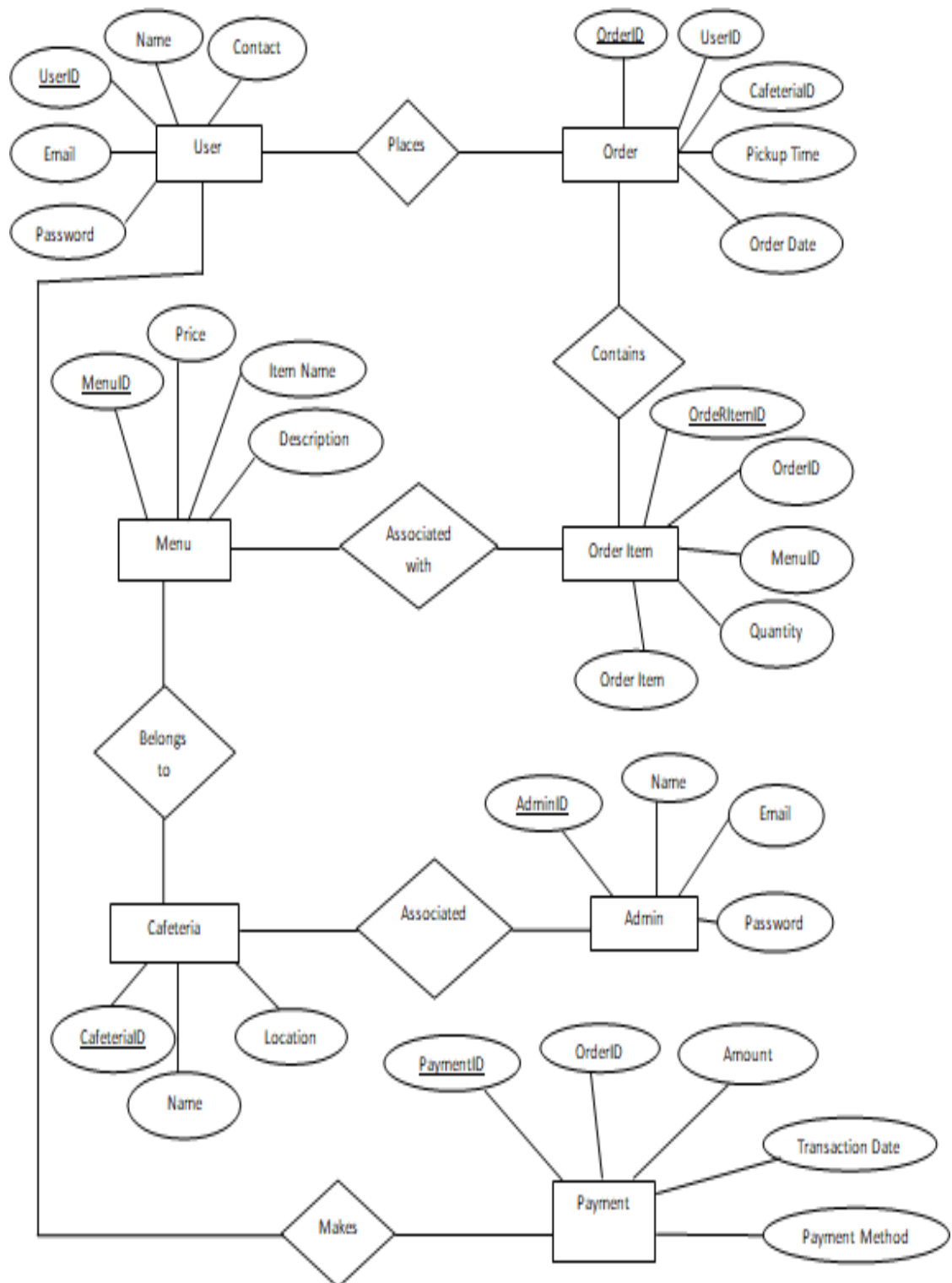


Fig.8. ER Diagram

Chapter 4: Implementation & Testing

Implementation & Testing

4.1 Methodology

4.1.1 Proposed Algorithm

To implement EazyEats effectively, we followed an Agile Development Methodology, which focuses on iterative development, frequent testing, and continuous feedback. The system was broken into small modules such as user login, menu management, payment, and order tracking — each developed and tested in cycles (sprints).

4.1.1.1 User Authentication:

- Input: Email and password
- Process: Verify credentials using Firebase Authentication
- Output: Access granted or error message

4.1.1.2 Menu Browsing & Pre-order:

- Input: Selected food items, quantity, time slot
- Process: Fetch menu data from the database, schedule order
- Output: Confirmed order details

4.1.1.3 Payment Integration:

- Input: User's payment option
- Process: Redirect to Razorpay, verify transaction
- Output: Payment success/failure message

4.1.1.4 Real-Time Order Tracking:

- Input: Order ID
- Process: Fetch order status updates from backend

- Output: Status displayed (e.g., Preparing, Ready)

4.2 Implementation Approach

4.2.1 Introduction to Languages, IDEs Tools and Technologies

The system was developed in modules using modern web technologies and tools. Frontend, backend, database, and integration services were carefully selected to support scalability, security, and ease of maintenance

TABLE 2 TOOLS & TECHNOLOGIES

Component	Tools & Technologies
Frontend	React.js, HTML5, CSS3, JavaScript, Bootstrap
Backend	Node.js, Express.js
Database	Firebase Firestore (NoSQL), MongoDB (optional for scalability)
Authentication	Firebase Authentication
Payment Gateway	Razorpay
Cloud Hosting	AWS
IDE Used	VS Code
Testing Tools	Postman (for API), Browser Dev Tools, Firebase Emulator Suite

4.3 Testing Approaches

4.3.1 Unit Testing

Each module (login, menu load, payment) was tested independently using manual and automated test cases.

a. Test Cases

TABLE 3 TEST CASES

Test Case ID	Module	Input	Expected Output	Result
TC_01	Login	Valid email & password	Successful login	Pass
TC_02	Login	Invalid credentials	Error message	Pass

TC_03	Menu Load	-	List of items displayed	Pass
TC_04	Order Placement	Item, Quantity, Time	Order Confirmed	Pass
TC_05	Payment	Valid Card/UPI	Payment Successful	Pass

4.3.2 Integration Testing

Integration testing was done to validate how modules interact, especially between frontend, backend, and external services like Razorpay.

b. Test Cases

TABLE 4 TEST CASES

Test Case ID	Scenario	Modules Involved	Expected Output	Result
IT_01	Complete Order	Login → Menu → Order → Payment	Order Placed ssfully	Pass
IT_02	Inventory Update	Order → Admin Dashboard	Updated Stock Count	Pass
IT_03	Notification r	Order Status Change	Real-time Notification	Pass
IT_04	Payment Failure	Razorpay → Backend → User	Payment Failed ge	Pass

Chapter 5: Results & Discussion

Results & Discussion

5.1 User Interface Representation

5.1.1 Brief Description of Various Modules

TABLE 5 MODULES

Module	Description
User Registration/Login	Allows users to create an account or sign in securely using Firebase authentication.
Menu Browsing	Displays daily cafeteria menu with categories, item descriptions, and price.
Pre-order System	Enables users to choose food items and schedule pickup time to queues.
Payment Integration	Seamless payment via Razorpay (UPI, cards, wallets).
Order Tracking	Users can view real-time status updates of their orders.
Admin Dashboard	Cafeteria staff can add/edit menu items, view orders, and monitor tory.
Inventory Management	Automatically updates stock levels after each order.
Notification System	Sends real-time alerts to users on order status or new menu items.

5.2 Snapshot of System with Brief Description

This section includes screenshots of different pages in the EazyEats system along with a brief description of each.

5.2.1 Login Page: Allows users and admins to securely log in to access the system features.

5.2.2 User Dashboard: Displays a welcome message and quick access to menu, order history, and tracking.

5.2.3 Menu Page: Shows all available food items with images, prices, and "Add to Cart" buttons.

5.2.4 Cart & Order Summary: Users can review selected items, adjust quantities, and confirm the order.

5.2.5 Payment Page: Offers multiple secure payment options like UPI, cards, and wallets.

5.2.6 Order Tracking: Lets users track their order status in real-time, such as "Preparing" or "Ready."

5.2.7 Admin Dashboard: Admins can manage menu items, view and update orders, and monitor sales.

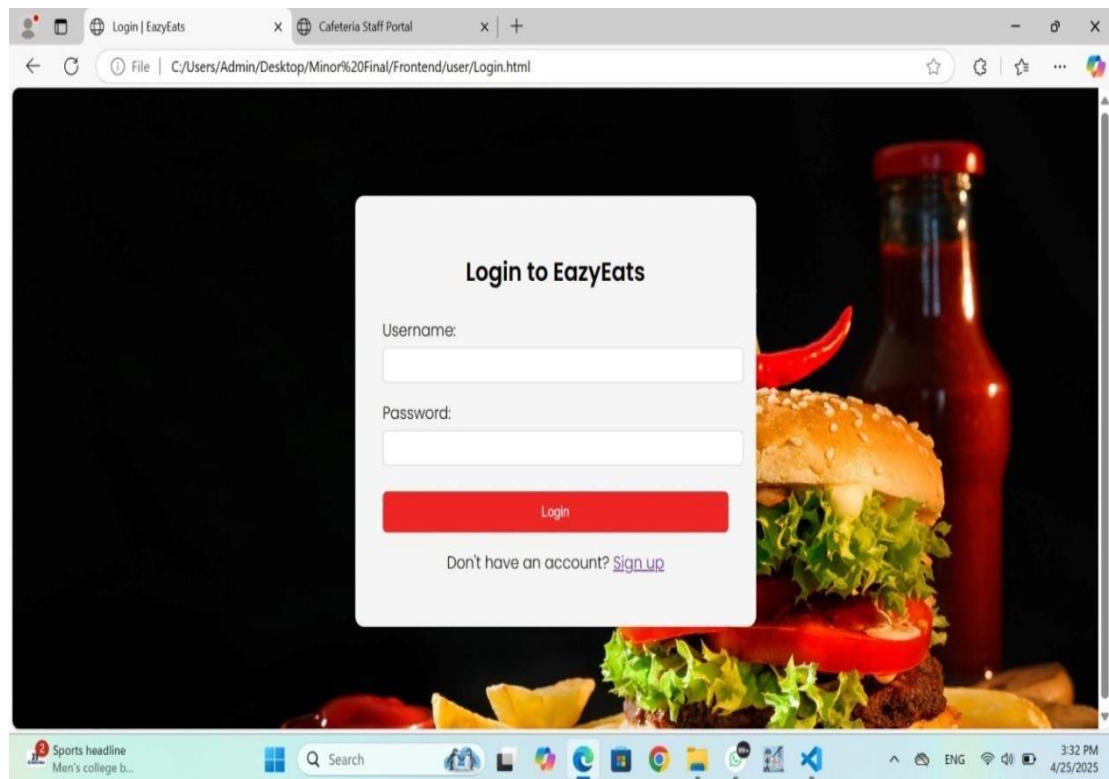


Fig.9. Login Page

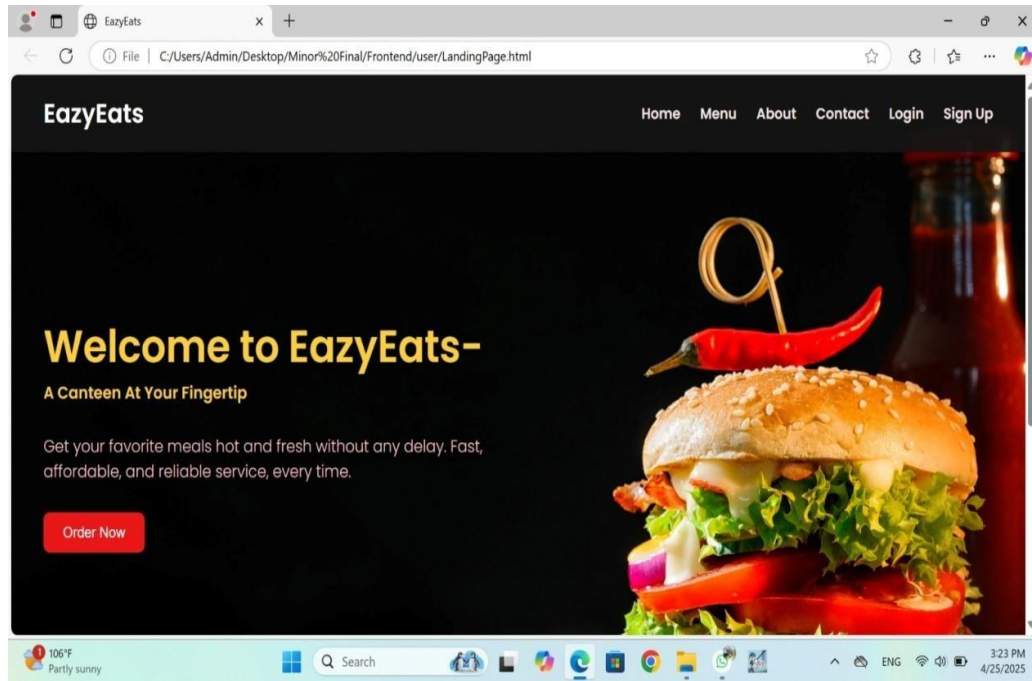


Fig.10. User Dashboard

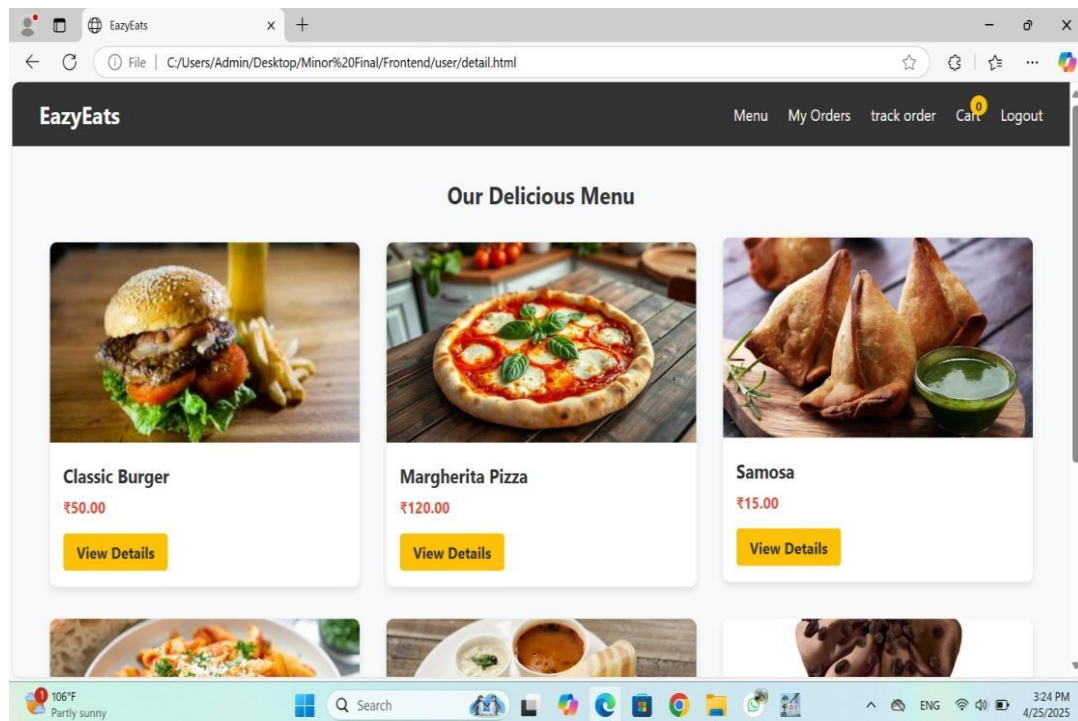


Fig.11. Menu Page

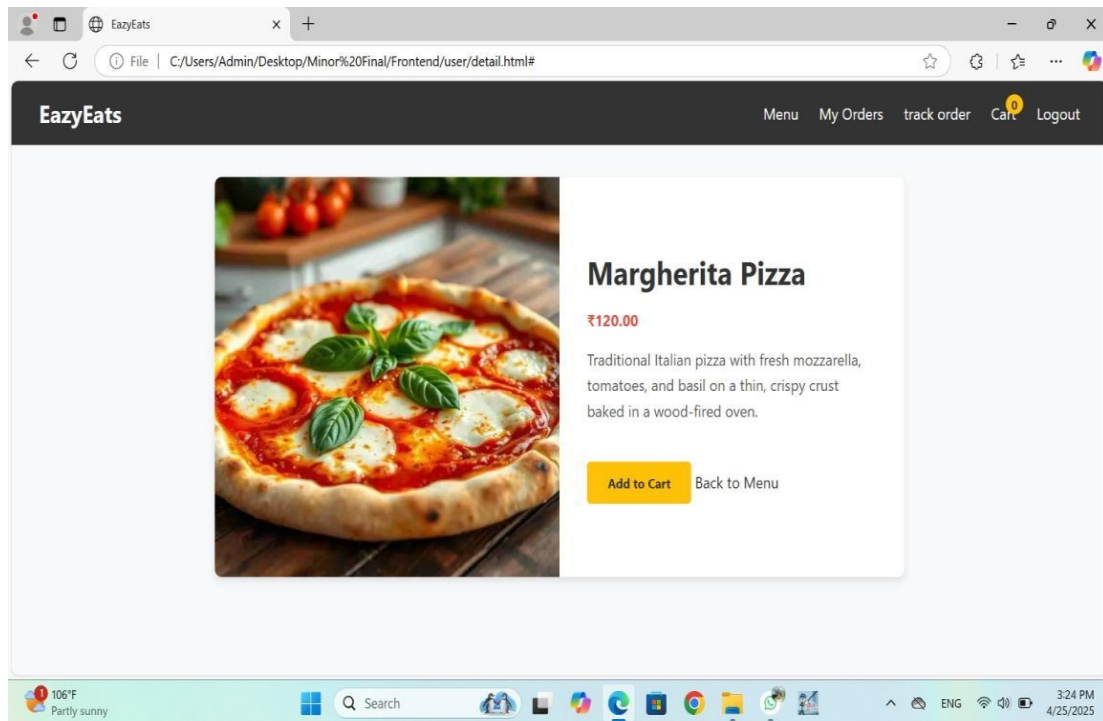


Fig.12. Add to Cart

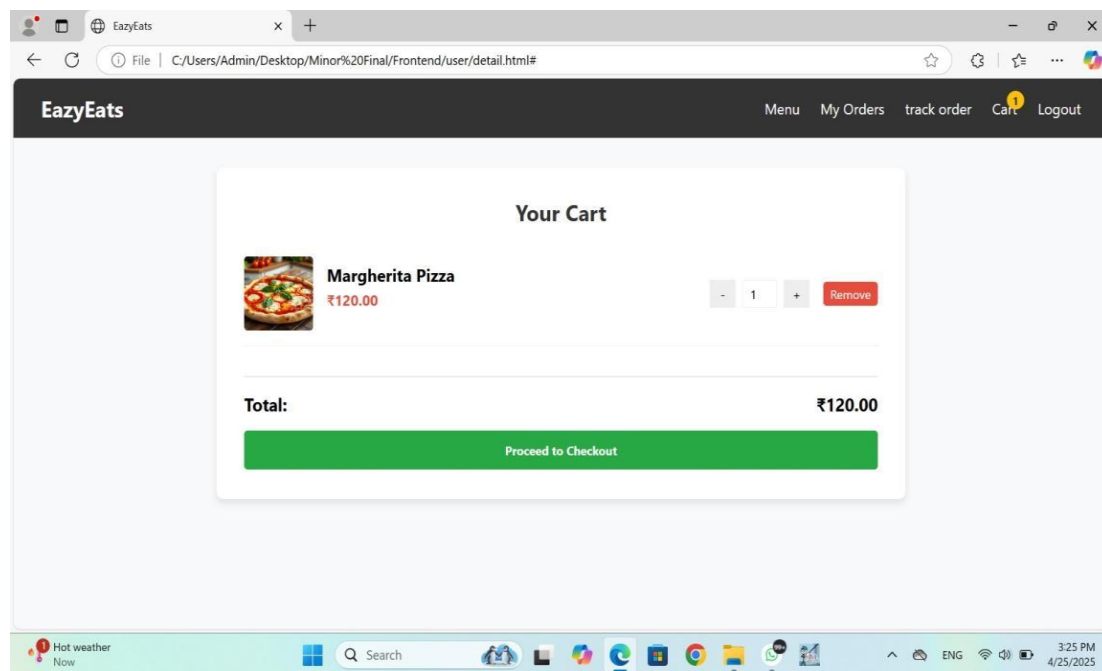


Fig.13.Order Summary

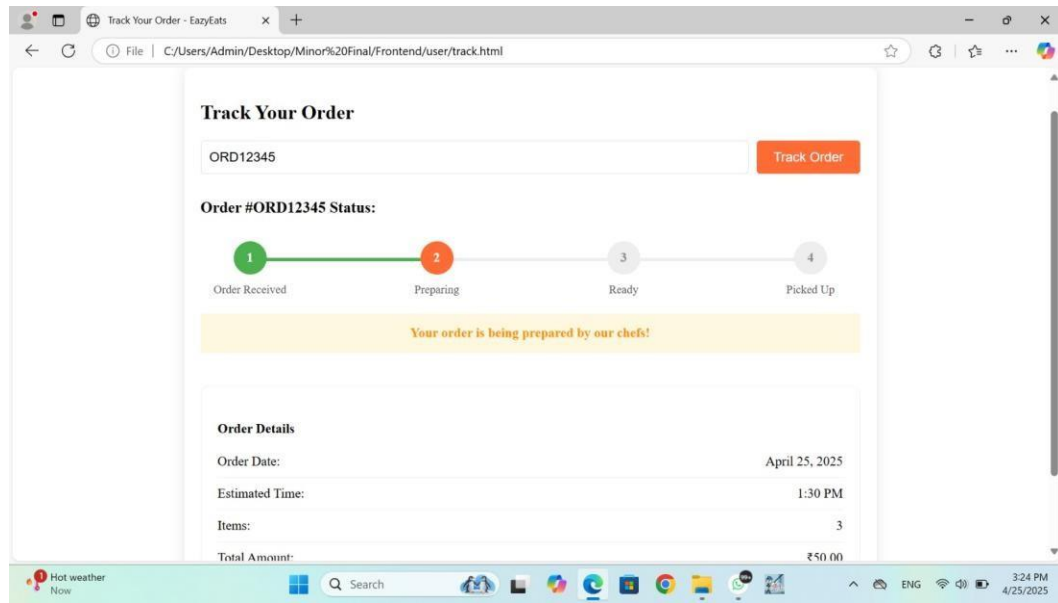


Fig.14. Order Tracking

5.3 Database Description

The EazyEats database is managed using Firebase Firestore, a cloud-hosted NoSQL database that stores data in the form of collections and documents. This structure is highly flexible and supports real-time synchronization, making it ideal for dynamic food ordering applications like EazyEats. The database includes several key collections such as **Users**, which stores user details including name, email, phone number, role (customer, staff, admin), and creation timestamp. The **Orders** collection records customer orders along with item details, total amount, order status, and order time. Payments are handled through a separate collection, storing information like payment method, amount, status, and payment timestamp. Additionally, the **Inventory** collection keeps track of available food items, their quantity, unit of measurement, and last updated time. An optional **Menu** collection can also be maintained to dynamically manage food items offered, including names, prices, and availability. Firestore's real-time data syncing allows instant updates across the application, supporting live order tracking for customers, up-to-date inventory for staff, and seamless interactions between users and the system.

5.3.1 Snapshot of Database Tables with Brief Description

5.3.1.1 Users Table

Stores user information such as name, email, phone number, and hashed password. It includes a role field to distinguish between customers, staff, and admins. Each user is uniquely identified by `user_id`.

5.3.1.2. Menu Items Table

Holds details about all available food items including name, description, price, category (like Breakfast or Lunch), and availability. Each item is uniquely identified by `item_id`.

5.3.1.3. Orders Table

Tracks every order placed by users. It links to the user who placed it, records the total amount, order status (e.g., pending, ready), and payment status. Each order has a unique `order_id`.

5.3.1.4. Payments Table

Stores payment details including method (e.g., UPI, card), amount, status (success, failed), and timestamp. It connects to the related order via `order_id`.

5.3.1.5. Inventory Table

Tracks stock of ingredients or items in the kitchen. It stores name, quantity, unit (like kg or pcs), and auto-updates the last modified time

TABLE 6 DATABASE

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
quantity	int	YES		NULL	
unit	varchar(20)	YES		NULL	
last_updated	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
email	varchar(100)	YES	UNI	NULL	
phone_number	varchar(20)	YES		NULL	
password_hash	varchar(255)	YES		NULL	
role	enum('customer','staff','admin')	YES		customer	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Field	Type	Null	Key	Default	Extra
item_id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
description	text	YES		NULL	
price	decimal(10,2)	YES		NULL	
category	varchar(50)	YES		NULL	
image_url	varchar(255)	YES		NULL	
available	tinyint(1)	YES		1	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
order_id	int	YES	MUL	NULL	
payment_method	varchar(50)	YES		NULL	
amount	decimal(10,2)	YES		NULL	
status	enum('success','failed','pending')	YES		pending	
payment_time	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

5.4 Final Findings

5.4.1 The EazyEats platform successfully digitizes and optimizes the cafeteria ordering system.

5.4.2 Users were able to place and track orders seamlessly, reducing waiting times and improving satisfaction.

5.4.3 Admins could efficiently manage menus and inventory, reducing food wastage.

5.4.4 Integration with Razorpay enabled smooth, secure, and contactless payments.

5.4.5 The real-time features, such as order tracking and notifications, enhanced operational transparency.

5.4.6 The system is scalable and can be deployed in universities, hospitals, offices, or food courts.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The development of EazyEats: A Canteen at Your Fingertips successfully addresses the common challenges faced in traditional cafeteria systems, such as long queues, delayed service, and food mismanagement. By leveraging cloud technologies and modern web development tools, EazyEats provides a seamless digital platform for users to pre-order meals, track their orders in real-time, and make secure online payments. Cafeteria staff benefit from simplified order management, dynamic menu control, and inventory tracking through the admin dashboard. The system not only enhances operational efficiency but also improves customer satisfaction and minimizes food wastage. Thus, EazyEats serves as an efficient, scalable, and user-friendly solution for modern cafeterias in educational institutions, offices, and other large facilities

6.2 Future Scope

EazyEats has the potential for further enhancement and scalability in the following ways:

6.2.1 Mobile App Integration: Developing native Android and iOS applications for improved accessibility and push notification support.

6.2.2 AI-Powered Recommendations: Implementing a recommendation engine to suggest meals based on user preferences and order history.

6.2.3 Multi-Vendor Support: Expanding the platform to support multiple cafeterias or food vendors within the same institution or complex.

6.2.4 Advanced Analytics: Adding detailed analytics for cafeteria staff to view customer trends, peak hours, and inventory predictions.

6.2.5 Voice Assistant Support: Integrating with voice assistants like Google Assistant or Alexa for hands-free ordering.

6.2.6 Loyalty Programs: Introducing reward points and discounts to increase user engagement and satisfaction.

6.2.7 Multi-Language Support: Making the system more inclusive by adding regional language options.

REFERENCES

1. Siddhant Sharma, Kirti Kushwah, Mayank Negi, Avi Sharma. “CafeEase:Improving College Cafeteria Experiences.” International Journal of Novel Research and Development (IJNRD), Vol. 9, Issue 4, pp. a790–a794, April 2024
2. Sidharth Shivam Singh, Vishal Ohal, Harsh Mittal, Aman Bansal, Swati Kashyap. “Cafeteria Management System - Enhancing its Efficiency.” International Journal for Research in Applied Science and Engineering Technology (IJRASET), 2023.
3. “Development and Evaluation of a Comprehensive Web-Based Canteen Food Ordering System.” International Journal for Research in Applied Science and Engineering Technology (IJRASET), 2023
4. “Online Catering System for College Canteen using Firebase.” International Journal for Research in Applied Science and Engineering Technology (IJRASET), 2023
5. “Cafeteria Food Ordering System using QR Code.” International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET), 2023
6. Ganadhish Navelkar, Bidesh Chanda, Vaibhav Sharma, Atish Gracias, Basil Jose, Valerie Menezes. “Canteen and Cafeteria Management System Using RFID Technology.” International Journal of Research Publication and Reviews (IJRPR), Vol. 5, Issue 7, July 2023

Appendix A: Project Synopsis

Appendix B: Guide Interaction Report

Appendix C: User Manual

Appendix D: Git/GitHub Commits/Version History

EasyEats/ at main · Palak016/EasyEats

github.com/Palak016/EasyEats/tree/main

Files

main

Go to file

Frontend/Html

images

servlets

AddItemServlet.java

CheckServlet.java

DeleteItemServlet.java

EditItemServlet.java

LoginServlet.java

PlaceOrderServlet.java

RegisterUserServlet.java

UpdateStatusServlet.java

README.md

addItem.jsp

admin.jsp

allorders.jsp

EasyEats /

Palak016 Create admin.html 425a918 · 7 minutes ago History

Name	Last commit message	Last commit date
Frontend/Html	Create admin.html	7 minutes ago
images	Create landingpageimage	9 minutes ago
servlets	Create UpdateStatusServlet.java	13 minutes ago
README.md	Initial commit	30 minutes ago
addItem.jsp	Create addItem.jsp	28 minutes ago
admin.jsp	Create admin.jsp	27 minutes ago
allorders.jsp	Create allorders.jsp	27 minutes ago
checkNewOrders.jsp	Create checkNewOrders.jsp	26 minutes ago
edititem.jsp	Create edititem.jsp	25 minutes ago
landingpage.jsp	Create landingpage.jsp	25 minutes ago
login.jsp	Create login.jsp	24 minutes ago
logout.jsp	Create logout.jsp	24 minutes ago

Type here to search

30°C 11:45 22-05-2025

EasyEats/ at main · Palak016/EasyEats

github.com/Palak016/EasyEats/tree/main

Files

main

Go to file

Frontend/Html

images

servlets

AddItemServlet.java

CheckServlet.java

DeleteItemServlet.java

EditItemServlet.java

LoginServlet.java

PlaceOrderServlet.java

RegisterUserServlet.java

UpdateStatusServlet.java

README.md

addItem.jsp

admin.jsp

allorders.jsp

EasyEats /

allorders.jsp Create allorders.jsp 22 minutes ago

checkNewOrders.jsp Create checkNewOrders.jsp 21 minutes ago

edititem.jsp Create edititem.jsp 20 minutes ago

landingpage.jsp Create landingpage.jsp 20 minutes ago

login.jsp Create login.jsp 19 minutes ago

logout.jsp Create logout.jsp 19 minutes ago

menu.jsp Create menu.jsp 18 minutes ago

myorders.jsp Create myorders.jsp 18 minutes ago

register.jsp Create register.jsp 17 minutes ago

README.md

EasyEats

EasyEats is a cloud-based food ordering system designed to streamline cafeteria operations and enhance the customer experience. Built for schools, colleges, offices, hospitals, and event venues, it addresses common issues such as long queues, delayed service, and order mismanagement.

Type here to search

30°C Sunny 11:40 22-05-2025